

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 4: C Generics, Endianness, Strings

Instructor: Justin Yokota

Agenda

- Strings
- C Memory model
- Stack
- Heap
- C Demos
- Alignment and Endianness
- Function Pointers
- Void Pointers

Agenda

- **Strings**
- C Memory model
- Stack
- Heap
- C Demos
- Alignment and Endianness
- Function Pointers
- Void Pointers

ASCII

- Unlike with numbers, there's no “natural” way to assign bitstrings to characters
- Current standard: ASCII
- Each character uses 8 bits of data (one byte)
- C does not distinguish between chars and 8-bit integers
 - Ex. The character '2' corresponds to ASCII value 50. If we did '2' + '2', C would interpret this as $50+50 = 100$, which, if turned back into a char, would be the character 'b'.
- The next slide will have a shortened version of the ASCII table; this will be provided whenever ASCII translations are needed.

ASCII table

HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR
0x20	32	SPACE	0x30	48	0	0x40	64	@	0x50	80	P	0x60	96	`	0x70	112	p
0x21	33	!	0x31	49	1	0x41	65	A	0x51	81	Q	0x61	97	a	0x71	113	q
0x22	34	"	0x32	50	2	0x42	66	B	0x52	82	R	0x62	98	b	0x72	114	r
0x23	35	#	0x33	51	3	0x43	67	C	0x53	83	S	0x63	99	c	0x73	115	s
0x24	36	\$	0x34	52	4	0x44	68	D	0x54	84	T	0x64	100	d	0x74	116	t
0x25	37	%	0x35	53	5	0x45	69	E	0x55	85	U	0x65	101	e	0x75	117	u
0x26	38	&	0x36	54	6	0x46	70	F	0x56	86	V	0x66	102	f	0x76	118	v
0x27	39	'	0x37	55	7	0x47	71	G	0x57	87	W	0x67	103	g	0x77	119	w
0x28	40	(	0x38	56	8	0x48	72	H	0x58	88	X	0x68	104	h	0x78	120	x
0x29	41	)	0x39	57	9	0x49	73	I	0x59	89	Y	0x69	105	i	0x79	121	y
0x2A	42	*	0x3A	58	:	0x4A	74	J	0x5A	90	Z	0x6A	106	j	0x7A	122	z
0x2B	43	+	0x3B	59	;	0x4B	75	K	0x5B	91	[	0x6B	107	k	0x7B	123	{
0x2C	44	,	0x3C	60	<	0x4C	76	L	0x5C	92	\	0x6C	108	l	0x7C	124	
0x2D	45	-	0x3D	61	=	0x4D	77	M	0x5D	93	]	0x6D	109	m	0x7D	125	}
0x2E	46	.	0x3E	62	>	0x4E	78	N	0x5E	94	^	0x6E	110	n	0x7E	126	~
0x2F	47	/	0x3F	63	?	0x4F	79	O	0x5F	95		0x6F	111	o	0x00	0	NULL

Strings

- Strings in C are defined as arrays of characters, similar to Python.
 - There isn't a primitive type called "string" in C. C strings are "char[]"s or "char*"s.
- Problem: C arrays don't know their own length (especially as pointers).
 - If we want to print out a string, we don't know when to stop printing...
- How to fix?
 - Approach 1: Store a length attribute (like Java does)
 - Approach 2 (what C does): Add a special "END" character to the end of the string
- For C, we use byte 0 ('\0', NULL) to represent the end of a string. This is known as the **null terminator**.

Strings

- For C, we use byte 0 ('\0', NULL) to represent the end of a string. This is known as the null terminator.
- Ex. The string “Hi” is stored in memory as an array of characters. This array would look like: { 'H', 'i', '\0' }, or in number form, {72, 105, 0}
- Strings need one more byte of memory than their length to be stored.
- Any string literals you use in your code will add a null terminator for you.
- However, if you create a string character-by-character in an array, you need to remember to add the null terminator yourself. **If the null terminator is missing, C will keep reading.**

Agenda

- Strings
- **C Memory model**
- Stack
- Heap
- C Demos
- Alignment and Endianness
- Function Pointers
- Void Pointers

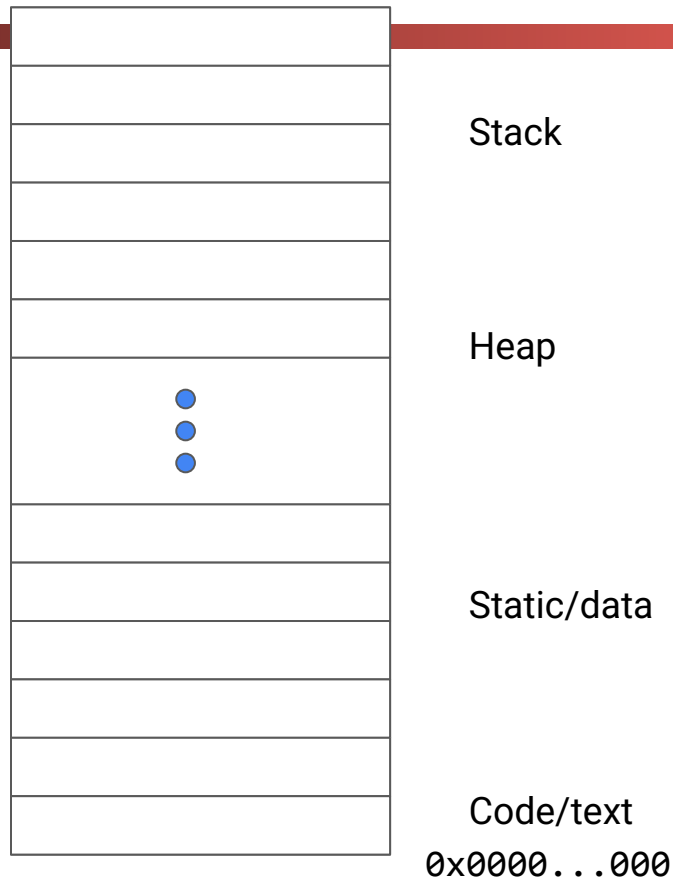
Memory Structure

0xFFFF...FFF

CS 61C

Summer 2026

- Main memory is a large array of bytes.
- When running a C program, data is stored in four main "chunks" of memory:
 - Stack
 - Heap
 - Static/data
 - Text/Code



Memory Structure

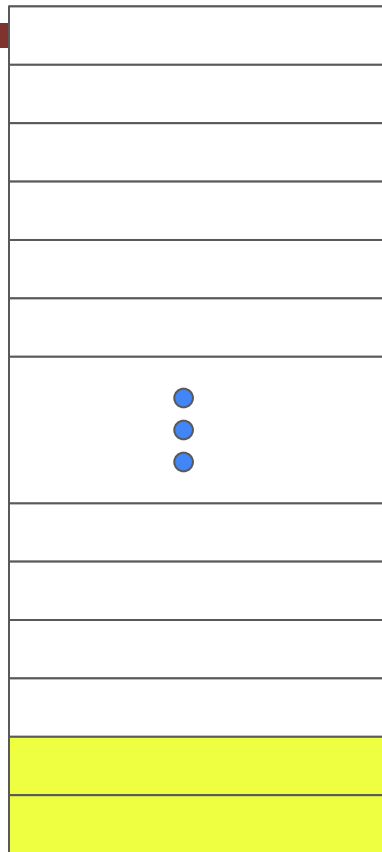
0xFFFF...FFF

CS 61C

Summer 2026

Code/text

- Executables are data too!
 - Big improvement over early computers (when programs were physical wire configurations)
- The code segment stores the actual bytecode that comprises your program.
- Fixed size, ideally never changed after loading the program
- Includes some constants!
 - Constants that are considered "built-in" to the code
 - Ex. $x = y + 1$; the 1 is part of the code.



0x0000...000

Memory Structure

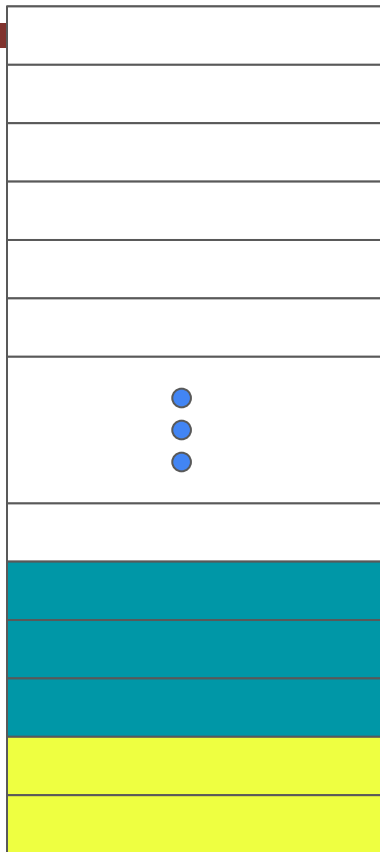
0xFFFF...FFF

CS 61C

Summer 2026

Static/data

- Stores anything that's a fixed size:
 - Global variables (you only ever need one copy of a global variable, so fixed size)
 - String literals (you can determine how many string literals there are in a program)



0x0000...000

Memory Structure

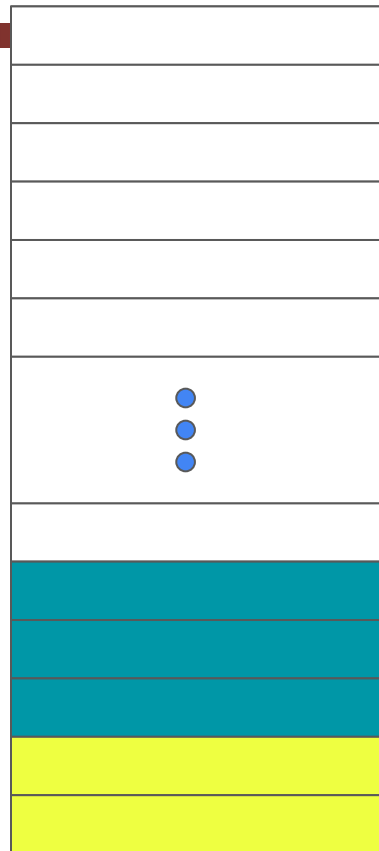
0xFFFF...FFF

CS 61C

Summer 2026

Static/data

- Every unique string gets stored once.
- If you have the same string several times in the code, only one copy gets saved (everything points to that copy).
- Corollary: String literals are **immutable** if they are stored in data.
 - `char* i = "Hi";`
 `i[0]="P"; // Error`
 `char* j = "Hi";`
- Only applies to string literals which need pointers!
- Ex. `char[] i = "Hi";` doesn't need a pointer to "Hi", so it treats the string the same as an integer literal.



0x0000...000

Agenda

- Strings
- C Memory model
- **Stack**
- Heap
- C Demos
- Alignment and Endianness
- Function Pointers
- Void Pointers

Memory Structure

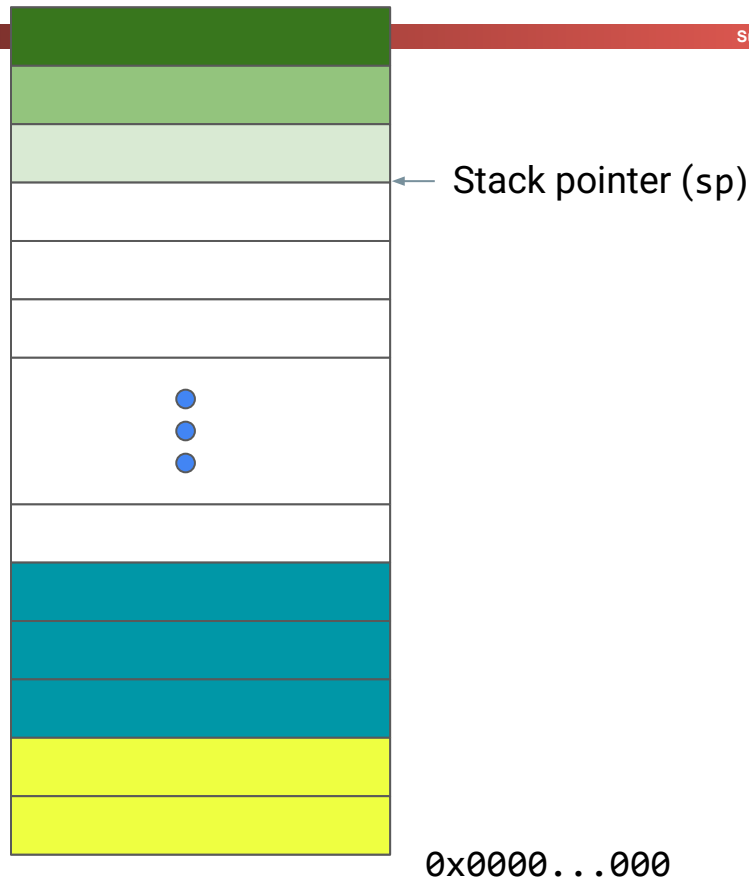
0xFFFF...FFF

CS 61C

Summer 2026

Stack

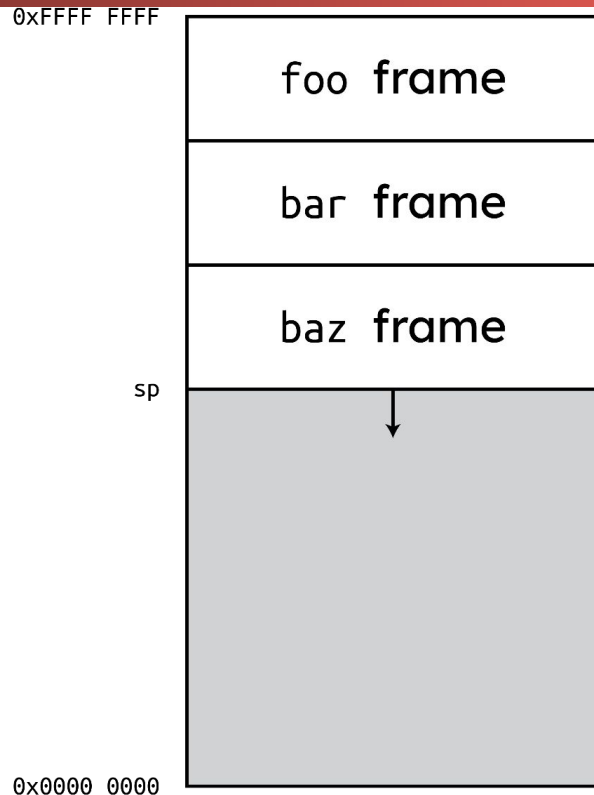
- Acts similarly to 61A "Environment Diagrams"
- Every function call sets aside some space on the stack for its local variables (plus some metadata)
- Designed for **temporary** storage; after a function returns, all data in the stack frame gets freed.
- Variable size; grows downward as you call functions and shrinks as you return from functions.
- All local variables are defined on the stack



Stack Frames

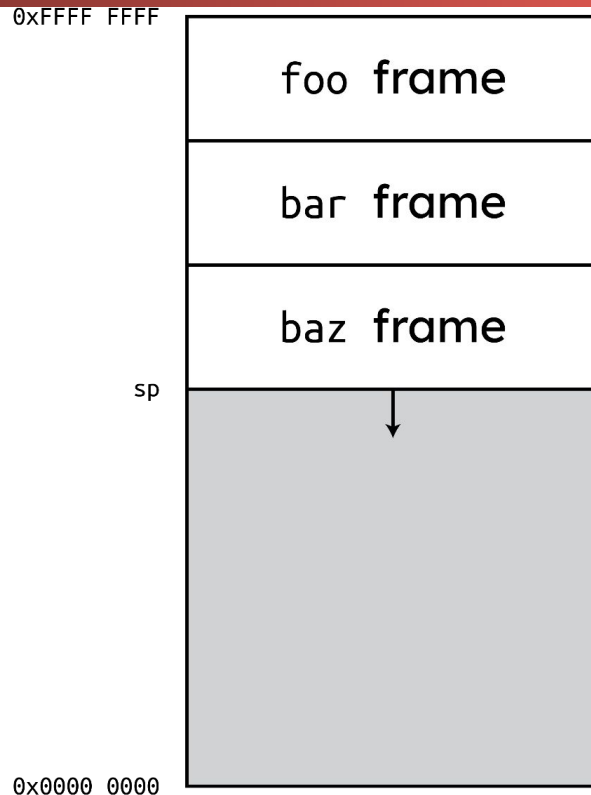
- Stack frames form a contiguous block of memory
- The bottom of the stack is marked by a **stack pointer**
- Example:

```
foo() { bar(); }  
bar() { baz(); }  
baz() { ... }
```



Common C Bug: Passing Stack Pointers

- What if bar returns a pointer to its local variable?
- What if foo gives bar a pointer to its local variable?
- Key idea:
 - Passing pointers into the stack is fine in one direction (caller to callee)
 - And dangerous in the other (callee to caller)



Agenda

- Strings
- C Memory model
- Stack
- **Heap**
- C Demos
- Alignment and Endianness
- Function Pointers
- Void Pointers

Memory Structure

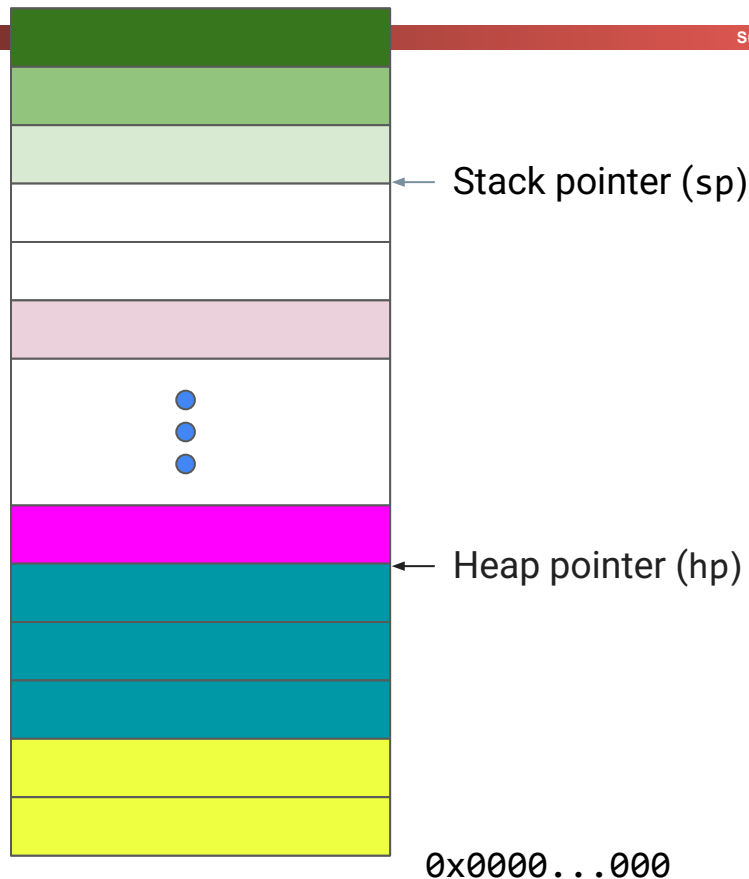
0xFFFF...FFF

CS 61C

Summer 2026

Heap

- Stack variables disappear when the function returns
- For any variables that need to persist across functions, use the heap
- In C, all heap memory must be manually allocated.
- Similarly, heap memory is only freed manually. **No garbage collector.**
- Grows upward as memory is allocated, and shrinks as memory is freed
- Big source of bugs



Heap

- Has nothing to do with the data structure
- The heap is the most flexible part of memory, but it's also the part that causes the most problems
 - Used similarly to Java's `new` command, but you need to do everything manually.
- Huge pool of memory (usually >> stack) **not** allocated in contiguous order.
- The heap is run by a *memory manager*, which is part of the *operating system* (or OS). If you want to use the heap, you need to ask for permission from the manager.

malloc

- `void* malloc(size_t n)`
 - Requests `n` consecutive bytes from the memory manager. The memory manager finds a block of open space and reserves that space for your personal use. This is known as a *Memory ALLOCation*.
 - The manager doesn't erase anything in that space; it's your responsibility to take out the garbage data
 - If the memory manager can't find enough memory (or you've used too much memory), returns a NULL pointer
 - Until you return the memory, the memory manager won't give that space to anyone else. It's yours.
- Analogy: the memory manager is a storage center. When you malloc, you ask the manager for a space, and they give you a key to a space that's big enough for you (but they don't clean that space out)

Heap Usage

- `void free(void* ptr)`
 - Gives the block pointed to by `ptr` back to the memory manager
 - `ptr` MUST be a pointer that was returned by the memory manager earlier that hasn't been freed; if you free a pointer to the stack, a pointer in the middle of an allocated block, or a pointer that was already freed, your code crashes
 - Does NOT clean out any data you put in the allocated block
- Analogy: Once you're done with that space, you return the key to the manager so they can give that space to someone else

Heap Usage

- `void* calloc(size_t nitems, size_t size)`
 - Allocates (`nitems * size`) bytes of data, and guarantees that the returned block is cleared (Clear ALLOCation)
 - Two main ways the memory manager does this:
 - Find a block that's already empty
 - Run `malloc`, then clear the block by zeroing out its contents
 - Either way, this makes `calloc` slightly slower than `malloc`
- Analogy: Same thing as `malloc`, but you pay a little extra to have the manager give you a storage space that's already empty

Heap Usage

- `void* realloc(void* ptr, size_t n)`
 - "Resizes" the block pointed to by `ptr`, so that it now contains `n` bytes.
 - `ptr` must be a pointer that was returned by the memory manager
 - Most of the time, it just resizes the box and returns the same pointer back. However, if it can't do so (ex. if the data immediately after the malloc'ed block is already allocated), the memory manager can choose to move the block to a new place in memory, freeing the old pointer.
 - Otherwise, acts as `malloc`; newly allocated memory contains garbage, and `realloc` returns `NULL` if it can't find a new block.
- Analogy: You need a larger-size storage space, so you give your current key back and get a new key to a bigger space. The manager moves your data to the new space for you. Alternatively, they tear down a wall and give you back your old key.

Heap Example

```
int* i = malloc(sizeof(*i) * 5);
for(int j = 0; j < 5; j++) i[j] = j;
int* k = calloc(3, sizeof(*k));
i = realloc(i, sizeof(*i)*2);
i = realloc(i, sizeof(*i)*10);
free(k);
k = malloc(sizeof(*k)*6);
for(int j = 0; j < 5; j++) k[j] = j;
free(i);
free(k+2);
```

Best Practices for Heap Use

- Return your memory before you forget
 - If you don't free a pointer before you lose track of the pointer, it'll stay allocated forever. This is known as a *memory leak*, and results in code that eventually breaks.
 - Frees are NOT recursive; if you have a complicated struct, free all components of the struct BEFORE you free the struct itself
- Use `sizeof` when deciding how much memory to allocate
 - You can use `sizeof(int)` in the previous example... but it makes the code less flexible if you later change `int` to something else.
- In most cases, the stack is faster to use (no need to search for memory), but also can't store as much data and can't store it long. Use the stack when you want something fast, and the heap when you want something last.

Agenda

- Strings
- C Memory model
- Stack
- Heap
- **C Demos**
- Alignment and Endianness
- Function Pointers
- Void Pointers

Code

- The code for these demos can be found here:
- Contains the following demos:
 - A comparison of how long malloc and calloc take
 - A test to see how a memory leak works, and when things crash
 - A test on reading uninitialized memory locations

Which is faster: Malloc or Calloc?

CS 61C

Summer 2026

Calloc is 2+ times faster

Calloc is faster by about 50%

Calloc is faster by about 10%

The two will be about the same speed (within 10% of each other's speeds)

Malloc is faster by about 10%

Malloc is faster by about 50%

Malloc is 2+ times faster



Which is faster: Malloc or Calloc?

Calloc is 2+ times faster

CS 61C

Summer 2026

Calloc is faster by about 50%

0%

Calloc is faster by about 10%

0%

The two will be about the same speed (within 10% of each other's speeds)

0%

Malloc is faster by about 10%

0%

Malloc is faster by about 50%

0%

Malloc is 2+ times faster

0%



Which is faster: Malloc or Calloc?

Calloc is 2+ times faster

CS 61C

Summer 2026

Calloc is faster by about 50%

0%

Calloc is faster by about 10%

0%

The two will be about the same speed (within 10% of each other's speeds)

0%

Malloc is faster by about 10%

0%

Malloc is faster by about 50%

0%

Malloc is 2+ times faster

0%



What will happen when this code is run?



CS 61C

Summer 2026

The code will run correctly forever

The code will run correctly for a long time, then suddenly crash

A compiler error

My computer will crash, and lecture ends early



What will happen when this code is run?



CS 61C

Summer 2026

0%

The code will run correctly for a long time, then suddenly crash

0%

A compiler error

0%

My computer will crash, and lecture ends early

0%



What will happen when this code is run?



CS 61C

Summer 2026

0%

The code will run correctly for a long time, then suddenly crash

0%

A compiler error

0%

My computer will crash, and lecture ends early

0%



What will happen when this code is run?



CS 61C

Summer 2026

A compiler error/warning

The code will crash

Almost all outputs are three characters long; "hi" followed by a random third char...

Almost all outputs are three characters or longer; "hi" followed by random nonse...

Almost all outputs are "hi", but some outputs are "hi" followed by random nonse...

All outputs are "hi"



What will happen when this code is run?



CS 61C

Summer 2026

- ☐ The code will crash 0%
- ☐ Almost all outputs are three characters long; "hi" followed by a random third character 0%
- ☐ Almost all outputs are three characters or longer; "hi" followed by random nonsense characters 0%
- ☐ Almost all outputs are "hi", but some outputs are "hi" followed by random nonsense characters 0%
- ☐ All outputs are "hi" 0%



What will happen when this code is run?



CS 61C

Summer 2026

- ☐ The code will crash 0%
- ☐ Almost all outputs are three characters long; "hi" followed by a random third character 0%
- ☐ Almost all outputs are three characters or longer; "hi" followed by random nonsense characters 0%
- ☐ Almost all outputs are "hi", but some outputs are "hi" followed by random nonsense characters 0%
- ☐ All outputs are "hi" 0%



The heap is a scary, scary place

- Code, Data — never change!
- Stack — managed automatically!
 - Just avoid returning pointers to local variables
- Heap — D:
 - Big 4 bugs:
 - “Memory leak” — forgetting to free
 - “Use after free” — use a pointer that has been freed
 - “Double free” — Calling free 2x on the same memory
 - “Realloc can move stuff” — Forgetting this

Memory Leak

If you forget to free, you will slowly run out memory. C **will not** help you!

- At first, your code works! Yay!
 - Memory leaks aren't a problem up to a critical point
- Then... “why is my entire computer so slow?”
 - Then, it gets bad
- Finally... your program dies
 - And then the OS stops you to save your computer



Use after Free

Also called “dangling reference”

- Using a pointer after its been freed can cause memory corruption
 - Best case: your program immediately crashes
 - Worse case: You end up on the NIST national security vulnerability database

CVE-2022-48771 Detail

RECEIVED

This vulnerability has been received by the NVD and has not been analyzed.

Description

In the Linux kernel, the following vulnerability has been resolved: drm/vmwgfx: Fix stale file descriptors on failed usercopy A failing usercopy of the fence_rep object will lead to a stale entry in the file descriptor table as put_unused_fd() won't release it. This enables userland to refer to a dangling 'file' object through that still valid file descriptor, leading to all kinds of use-after-free exploitation scenarios. Fix this by deferring the call to fd_install() until after the usercopy has succeeded.

Double Free

Only free something once!

- Best case: you corrupt your heap and you crash
 - Because you didn't free something returned by malloc
- Worst case: you free something able to be freed on accident
 - ... and end up on the NIST national vulnerability database.

CVE-2023-39975 Detail

MODIFIED

This vulnerability has been modified since it was last analyzed by the NVD. It is awaiting reanalysis which may result in further changes to the information provided.

Description

kdc/do_tgs_req.c in MIT Kerberos 5 (aka krb5) 1.21 before 1.21.2 has a double free that is reachable if an authenticated user can trigger an authorization-data handling failure. Incorrect data is copied from one ticket to another.

realloc

Don't forget to reassign!

- Best case: you're lucky and your data doesn't move
- Worst case: you get a dangling pointer
 - ... and do you see the pattern now?

CVE-2024-22088 Detail

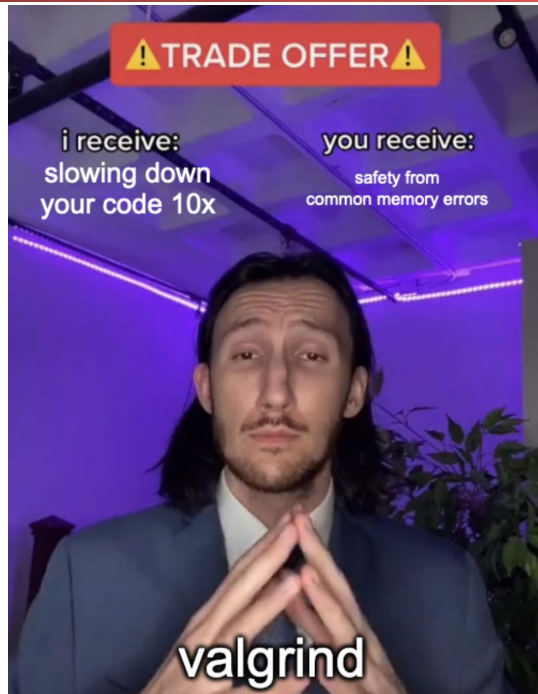
Description

Lotos WebServer through 0.1.1 (commit 3eb36cc) has a use-after-free in `buffer_avail()` at `buffer.h` via a long URI, because `realloc` is mishandled.

valgrind!!!

Valgrind is an essential part of a C coder's tool kit

- Valgrind slows down your code *a lot* (~10x or more)
- But... it catches:
 - Memory leaks
 - Misuse of free
 - Buffer overflows (writing past the end of arrays)



Agenda

- Strings
- C Memory model
- Stack
- Heap
- C Demos
- **Alignment and Endianness**
- Function Pointers
- Void Pointers

Alignment

- Many processors want to have “aligned memory”
 - Some will not allow you to access non-word aligned values
 - Others will be very slow
- In C, types also have alignment requirements
 - General rule: N-byte objects prefer to start at an address divisible by N

	+0	+1	+2	+3
0x100	int32_t			
0x104	int16_t	8_t	8_t	
0x108	8_t		int16_t	
0x10C	int32_t			
0x110				
0x114				
0x118				
0x11C				
...				

Alignment

Variables will automatically be put
in aligned addresses
In the stack, variables can be
reordered to save space

```
uint32_t x = 100;  
char c = 'A';  
int16_t y = 5;  
char d = '?';
```

		+0	+1	+2	+3
main Frame	uint32_t x	0x100			
		0x104	100		
	y	0x108	5	'A'	'?'
		0x10C			
		0x110			
		0x114			
		0x118			
		0x11C			
		...			

Struct Alignment

For structs:

1. Members must be in the order given, but padding can be added between members
2. Every member must have its alignment satisfied
3. The entire struct is aligned to its largest aligned member

```
1 struct foo {  
2     int8_t  a;  
3     int32_t b;  
4     int16_t c;  
5 };
```

What is `sizeof(struct foo)`?

Struct Alignment

For structs:

1. Members must be in the order given, but padding can be added between members
2. Every member must have its alignment satisfied
3. The entire struct is aligned to its largest aligned member

```
1 struct foo {  
2     int8_t a;  
3     int32_t b;  
4     int16_t c;  
5 };
```

What is `sizeof(struct foo)`?

Struct Alignment

For structs:

1. Members must be in the order given
2. Every member must have its alignment satisfied
3. The entire struct is aligned to its largest aligned member

```
1 struct foo {  
2     int8_t a;  
3     int32_t b;  
4     int16_t c;  
5 };
```

What is `sizeof(struct foo)`?

	+0	+1	+2	+3
0x100	a			
0x104		b		
0x108	c			
0x10C				
0x110				
0x114				
0x118				
0x11C				
...				

Struct Alignment

For structs:

1. Members must be in the order given
2. Every member must have its alignment satisfied
3. The entire struct is aligned to its largest aligned member

```
1 struct foo {  
2     int8_t a;  
3     int32_t b;  
4     int16_t c;  
5 };
```

What is `sizeof(struct foo)`? **12**

Can we make this better?

	+0	+1	+2	+3
0x100	a			
0x104		b		
0x108	c			
0x10C				
0x110				
0x114				
0x118				
0x11C				
...				

Struct Alignment

For structs:

1. Members must be in the order given
2. Every member must have its alignment satisfied
3. The entire struct is aligned to its largest aligned member

```
1 struct foo {  
2     int8_t a;  
3     int16_t c;  
4     int32_t b;  
5 };
```

Yes. If we swap b and c, we can reduce size to 8

	+0	+1	+2	+3
0x100	a		c	
0x104		b		
0x108				
0x10C				
0x110				
0x114				
0x118				
0x11C				
...				

Endianness

What does the following code print?

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n", arr[0]);  
printf("%c\n", (char*)arr);  
printf("%s\n", arr);
```

		+0	+1	+2	+3
uint32_t arr	0x100	0x64726167			
	0x104	0x73206E65			
	0x108	0x7400646F			
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

Endianness

The first line should print
0x64726167

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

		+0	+1	+2	+3
uint32_t arr	0x100	0x64726167			
	0x104	0x73206E65			
	0x108	0x7400646F			
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

Endianness

The second line reads one byte located at 0x100 as a char.
The specific data at that byte depends on how we split arr[0]

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n", arr[0]);  
printf("%c\n", (char*)arr);  
printf("%s\n", arr);
```

		+0	+1	+2	+3
uint32_t arr	0x100	0x64726167			
	0x104	0x73206E65			
	0x108	0x7400646F			
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

Big Endianness vs Little Endianness

Two main ways you can split an int into bytes:

Option 1 (Big-endian): Order the bytes such that the **most significant byte** (byte with the biggest effect) comes first

Option 2 (Little-endian): Order the bytes such that the **least significant byte** (byte with the smallest effect) comes first

<code>arr[0]</code>	+0	+1	+2	+3
0x100	0x64726167			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Big Endianness vs Little Endianness

Both approaches work, as long as you are consistent with how you read and write the data.

Analogy: How do you write dates?

- Europe: 26-6-2025 (little endian)
- China/Japan: 2025年6月26日 (big endian)
- US: 6/26/2025 (middle endian)

Mostly up to preference; everyone can tell the date regardless of how they write it

arr[0]	+0	+1	+2	+3
0x100	0x64726167			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Big Endianness vs Little Endianness

- Little-endian is common architectures (unless otherwise stated, assume little endian)
- Big-endian is common in network communications.
- Standardization is hard, okay?

arr[0]	+0	+1	+2	+3
0x100	0x64726167			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Big Endian vs Little Endian

Let's try running this same code on the two different endiannesses

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	0x64726167			
	0x104	0x73206E65			
	0x108	0x7400646F			

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	0x64726167			
	0x104	0x73206E65			
	0x108	0x7400646F			

Big Endian vs Little Endian

When we create the array, we set data to appropriate values. The data gets split by the compiler according to the endianness of the system (0x omitted)

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

`arr[0]` is an int, consisting of the bytes 0x100, 0x101, 0x102, and 0x103. We read those 4 bytes, and combine them to 1 int.

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Big endian: data is 64, 72, 61, 67

Combine the data with the first byte being most significant:

0x64726167

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Little endian: data is 67, 61, 72, 64

Combine the data with the first byte
being least significant:

0x64726167

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n", arr[0]);  
printf("%c\n", (char*)arr);  
printf("%s\n", arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Regardless of endianness, the int we write is the int we read. **Endianness only affects when we read a part of a block at a time.**

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

We read the byte at 0x100

For Big-endian: Byte is 0x64 = 'd'

For Little-endian: Byte is 0x67 = 'g'

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n", arr[0]);  
printf("%c\n", (char*)arr);  
printf("%s\n", arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Strings are char arrays, so we read bytes starting at 0x100, in increasing order, until we hit a 00 byte.

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Big endian: Sequence goes 64,
72, 61, 67, 73, 20, 6E, 65, 74, 00

= "drags net"

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Little endian: Sequence goes 67,
61, 72, 64, 65, 6E, 20, 73, 6F, 64,
00

= "garden sod"

```
uint32_t arr[] =  
{0x64726167,  
0x73206E65,  
0x7400646F};  
printf("%x\n",arr[0]);  
printf("%c\n",(char*)arr);  
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Endianness summary

Both endiannesses work within C; it's technically undefined behavior to split an int into chars like we did here.

Remember: writing and reading is always consistent

Common mistake students have is to "reverse order" when writing the bytes, but then forgetting to "reverse reading order" when reading bytes (especially in more complex cases).

<code>arr[0]</code>	+0	+1	+2	+3
0x100	0x64726167			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Agenda

- Strings
- C Memory model
- Stack
- Heap
- C Demos
- Alignment and Endianness
- **Function Pointers**
- Void Pointers

More Pointers

Pointers can point to more than just variables!

- Function Pointers:
 - `int *(*fp) (int, int)`
 - Point to functions
 - Lets us implement higher order functions
- Generic Pointers:
 - `void* p`
 - Points to *anything*
 - Lets us implement generic functions

Function Pointers

Function pointers are variables storing addresses of a function

- Declaration:

- `int>(*fp)(int, int) = &foo;`
- `int>(*fp)(int, int) = foo;     // Can only be omitted for function pointers`

- Usage:

- `(*fp)(x, y)`
- `fp(x, y)     // Can only be omitted for function pointers`

- Note: Convention

- Omitting `&` and `(*...)` follows the “less is more” ideology
- Leaving them in follows the “be clear” / “defensive programming” ideology

Example: map

```
1  /* Mutates an array ARR of length N by
2   * replacing each element of ARR with
3   * the result of calling FP on that element. */
4  void map(int arr[], size_t n, int (*fp)(int)) {
5      for (int i = 0; i < n; i++) {
6          arr[i] = fp(arr[i]); // remember that arr[] is a pointer!
7      }
8  }
9
10 int times2 (int x) { return 2 * x; }
11 int times10(int x) { return 10 * x; }
```

Example: Using map

```
1  #include <stdio.h>
2  #define ARR_LEN 3
3  int main() {
4      int arr[ARR_LEN] = {3, 1, 4};
5      printf("arr: [%d, %d, %d]\n", arr[0], arr[1], arr[2]);
6      map(arr, ARR_LEN, &times2);
7      printf("arr: [%d, %d, %d]\n", arr[0], arr[1], arr[2]);
8      map(arr, ARR_LEN, &times10);
9      printf("arr: [%d, %d, %d]\n", arr[0], arr[1], arr[2]);
10 }
```

```
$ gcc main.c && ./a.out
arr: [3, 1, 4]
arr: [6, 2, 8]
arr: [60, 20, 80]
```

Agenda

- Strings
- C Memory model
- Stack
- Heap
- C Demos
- Alignment and Endianness
- Function Pointers
- **Void Pointers**

Why use generic functions?

- Writing general-purpose code
 - Reduces code rewriting, avoids having to make the same change in multiple places
 - Used throughout C libraries for things like:
 - Free arbitrary memory
 - Sort any array
- In general,
 - Generics should work for any argument type
 - Updates blocks of memory regardless of data types

The `void*` pointer

- Doesn't mean “pointer to nothing” (yes the name is misleading)
- Instead means “pointer to anything”
- Quirks:
 - You can **never, ever** dereference a `void*` pointer
 - You can't do pointer arithmetic with `void*` pointers
 - Why?
 - `void*` pointers can be safely and automatically converted to other pointer types

```
void* ptr1;  
/* some code */  
int* ptr2 = ptr1;
```

- Note: `void*` pointers cannot be automatically cast to function pointers.

Danger: Dereferencing

Compare the two below blocks of code. What should happen?

```
1  void *ptr = ...;
2  printf("%p\n", *ptr); // Note: %p means "pointer"
```

```
1  void **doubleptr = ...;
2  printf("%p\n", *doubleptr);
```

Danger: Dereferencing

Compare the two below blocks of code. What should happen?

```
1 void *ptr = ...;
2 printf("%p\n", *ptr); // Note: %p means "pointer"
```

```
main.c:6:17: warning: ISO C does not allow indirection on operand of type 'void *'
             [-Wvoid-ptr-dereference]
             printf("%p\n", *ptr);
                        ^~~~
```

```
main.c:6:17: error: argument type 'void' is incomplete
```

```
1 void **doubleptr = ...;
2 printf("%p\n", *doubleptr);
```

- Perfectly fine!

- doubleptr is... **a pointer to...** a pointer to anything
- Also, we know `sizeof(void *)`! (It's the same size as any other pointer)

Example: C heap library

```
$ man malloc
```

```
NAME
    calloc, free, malloc, realloc, reallocf, valloc, aligned_alloc - memory allocation
```

```
SYNOPSIS
    #include <stdlib.h>

    void *
    calloc(size_t count, size_t size);

    void
    free(void *ptr);

    void *
    malloc(size_t size);

    void *
    realloc(void *ptr, size_t size);

    ...
```

Generic Memory Functions

We have two main functions to work with memory:

- `void* memcpy(void *dest, void *src, size_t count);`
 - Copies `count` bytes from `src` to `dest`.
 - Undefined behavior if `src` and `dest` overlap (i.e. `(char *) src + count >= dest`)
 - Fast!!
- `void* memmove(void *dest, void *src, size_t count);`
 - Copies `count` bytes from `src` to `dest`.
 - No problems if `src` and `dest` overlap (i.e. `(char *) src + count >= dest`)
 - Slow!!
- Both
 - Return a copy of `dest`
 - Fail if either `src` or `dest` are invalid or `NULL`.

Swapping

```
1 void swap_ints(int *x, int *y) {  
2     int tmp = *x;  
3     *x = *y;  
4     *y = tmp;  
5 }
```

```
1 void swap_floats(float *x, float *y) {  
2     float tmp = *x;  
3     *x = *y;  
4     *y = tmp;  
5 }
```

```
1 void swap_strings(char **x, char **y) {  
2     char *tmp = *x;  
3     *x = *y;  
4     *y = tmp;  
5 }
```

Swapping

```
1 void swap(void *x, void *y) {  
2     /* Save x in tmp  
3     * Copy value in y to x  
4     * Copy value in tmp to x */  
5 }
```

- Idea: reuse swapping code
- However...
 - How do we know the size of the arguments?
 - Let's add a new argument

Swapping

```
1  void swap(void *x, void *y, size_t nbytes) {  
2      /* Save x in tmp  
3      * Copy value in y to x  
4      * Copy value in tmp to x      */  
5  }
```

- Idea: reuse swapping code
- However...
 - How do we know the size of the arguments?
 - Let's add a new argument

Swapping — Full Code

```
1 void swap(void *x, void *y, size_t nbytes) {  
2     char tmp[nbytes];           // Create space to hold x;  
3     memcpy(tmp, x, nbytes);     // Save value of x in tmp  
4     memcpy(x, y, nbytes);       // Copy value in y to x  
5     memcpy(y, tmp, nbytes);     // Copy value in tmp to y  
6 }
```

- In C, `sizeof(char) == 1` by definition
 - i.e. a `char` is C's version of a byte
- Why can't we replace line 4 with `*x = *y`?

Swapping — Full Code

```
1 void swap(void *x, void *y, size_t nbytes) {  
2     char tmp[nbytes];           // Create space to hold x;  
3     memcpy(tmp, x, nbytes);     // Save value of x in tmp  
4     memcpy(x, y, nbytes);      // Copy value in y to x  
5     memcpy(y, tmp, nbytes);    // Copy value in tmp to y  
6 }
```

- In C, `sizeof(char) == 1` by definition
 - i.e. a `char` is C's version of a byte
- Why can't we replace line 4 with `*x = *y`?
 - Dereferencing `void*`!

Swapping — Demo

```
1  int main(int argc, char *argv[]) {  
2      int a = 6;  
3      int b = 1;  
4      printf("a: %d | b: %d\n", a, b);  
5      swap(&a, &b, sizeof a);  
6      printf("a: %d | b: %d\n", a, b);  
7  }
```

```
$ gcc main.c && ./a.out
```

```
a: 6 | b: 1
```

```
a: 1 | b: 6
```

Generic Functions on Arrays

How do we do pointer arithmetic on `void*` pointers? Cast first!

- Given:

```
void swap(void *x, void *y, size_t nbytes)
```

- Let's write the following function:

```
void swap_ends(void *arr, size_t nelems, size_t nbytes)
```

- What type should we use for pointer arithmetic?
 - Hint: what type is guaranteed to have size 1?

Swap Ends

```
1 void swap_ends(void *arr, size_t nelems, size_t nbytes) {  
2     void *start = arr;  
3     void *end   = (__a__) arr + __b__;  
4     swap(start, end, nbytes);  
5 }
```

- What type should we use for pointer arithmetic?
 - Hint: what type is guaranteed to have size 1?

Swap Ends

```
1 void swap_ends(void *arr, size_t nelems, size_t nbytes) {  
2     void *start = arr;  
3     void *end   = (char *) arr + __b__;  
4     swap(start, end, nbytes);  
5 }
```

- What type should we use for pointer arithmetic?
 - Hint: what type is guaranteed to have size 1?

Swap Ends

```
1 void swap_ends(void *arr, size_t nelems, size_t nbytes) {  
2     void *start = arr;  
3     void *end   = (char *) arr + __b__;  
4     swap(start, end, nbytes);  
5 }
```

- How should we do our pointer arithmetic?
 - Hint: Normally, we would do `&arr[nelems - 1]` to refer to the address of the last element
 - Hint 2: Remember that `arr[nelems - 1]` means `*arr + (nelems - 1)`
 - Hint 3: Pointer arithmetic in C automatically multiplies by the size of what is being pointed to, but we don't have that luxury in generic pointer arithmetic — you have to multiply by the size

Swap Ends — Full Code

```
1 void swap_ends(void *arr, size_t nelems, size_t nbytes) {  
2     void *start = arr;  
3     void *end   = (char *) arr + (nelems - 1) * nbytes;  
4     swap(start, end, nbytes);  
5 }
```

- How should we do our pointer arithmetic?
 - Hint: Normally, we would do `&arr[nelems - 1]` to refer to the address of the last element
 - Hint 2: Remember that `arr[nelems - 1]` means `*(arr + (nelems - 1))`
 - Hint 3: Pointer arithmetic in C automatically multiplies by the size of what is being pointed to, but we don't have that luxury in generic pointer arithmetic — you have to multiply by the size

Swap Ends — Test

```
1  #define ARR_LEN 4
2  int main(int argc, char *argv[]) {
3      int arr[ARR_LEN] = {1, 4, 3, 4};
4      printf("arr: [%d, %d, %d, %d]\n", arr[0], arr[1], arr[2], arr[3]);
5      swap_ends(arr, ARR_LEN, sizeof arr[0]);
6      printf("arr: [%d, %d, %d, %d]\n", arr[0], arr[1], arr[2], arr[3]);
7  }
```

```
$ gcc main.c && ./a.out
arr: [1, 4, 3, 4]
arr: [4, 4, 3, 1]
```